
Manual de instalación y configuración

Lyra WatchLog — guía de instalación paso a paso (Linux y Windows)

ITESICWS · Lyra WatchLog

Julio 2026

Índice

0. Antes de empezar (léelo, son 2 minutos)	2
0.1 Cómo funciona la instalación (idea general)	2
0.2 Las máquinas y roles que intervienen	2
0.3 Cómo leer los comandos (etiquetas de color)	2
0.4 Decisiones que tomarás durante la instalación	3
0.5 Anota estos datos (los usarás varias veces)	3
PARTE A — Instalar en LINUX	4
A1. Verifica que el servidor cumple los requisitos	4
A2. Conéctate al servidor e instala Docker	4
A3. Copia el paquete al servidor	5
A4. Desempaqueta el paquete	5
A5. Primera pasada del instalador	5
A6. Dile a la app su dirección (edita SOLO .env)	6
A7. Segunda pasada del instalador (enciende todo)	6
A8. Ceremonia de licencia	7
A9. Crea tu usuario administrador	7
A10. Entra y prueba	8
A11. Si algo sale mal (diagnóstico)	8
A12. Operación diaria (Linux)	8
PARTE B — Instalar en WINDOWS	9
B1. Verifica que el equipo cumple los requisitos	9
B2. Instala WSL2	9
B3. Instala Docker Desktop (en modo contenedores Linux)	9
B4. Instala OpenSSL	10
B5. Descarga y desempaqueta el paquete	10
B6. Primera pasada del instalador	10
B7. Dile a la app su dirección (edita SOLO .env)	11
B8. Segunda pasada del instalador (enciende todo)	11
B9. Licencia + administrador + prueba	11
B10. Si el probe de permisos falla (Windows)	12
B11. Si algo sale mal (diagnóstico Windows)	12
B12. Operación diaria (Windows)	12
Apéndice — Lista de verificación final (ambos sistemas)	13
Glosario mínimo	13
Para profundizar	13

Guía a prueba de todo, paso por paso, de CERO a funcionando. Está pensada para que la siga **cualquier persona**, aunque no sea experta: solo tienes que hacer los pasos **en orden** y **no saltarte ninguno**. Hay **dos recorridos completos y separados**:

- **PARTE A — Instalar en LINUX** (destino principal, producción).
- **PARTE B — Instalar en WINDOWS** (Docker Desktop / WSL2; pilotos/estaciones).

Elige **UNA** parte según tu servidor. No mezcles las dos. Cuando termines la que te corresponde, la aplicación quedará **instalada, con licencia y operativa**.

0. Antes de empezar (léelo, son 2 minutos)

0.1 Cómo funciona la instalación (idea general)

Lyra WatchLog se entrega como **un solo archivo comprimido** (.tar.gz) que contiene TODO lo necesario (la aplicación, la base de datos, todo) para funcionar **sin internet**. Tú lo copias al servidor, corres **un instalador** y él hace el resto. No necesitas descargar nada más ni saber de programación.

0.2 Las máquinas y roles que intervienen

Quién	Máquina	Para qué
Tú (operador)	El servidor del cliente (Linux o Windows)	Correr el instalador y dejar la app arriba.
Tú (operador)	Tu PC (Windows, normalmente)	Conectarte al servidor y mover 2 archivos de licencia.
El proveedor (ITESICWS)	Su PC con la llave de emisión	Generar el archivo de licencia a partir de tu solicitud.

El servidor **nunca** necesita internet. Solo viajan **dos archivos pequeños** de licencia (uno de ida, uno de vuelta).

0.3 Cómo leer los comandos (etiquetas de color)

Cada bloque de comandos tiene una etiqueta que te dice **DÓNDE** escribirlo:

Etiqueta	Dónde
TU PC (PowerShell)	Tu Windows de siempre. Menú Inicio → escribe PowerShell → ábrelo.
SERVIDOR LINUX (bash)	Dentro del servidor Linux, tras conectarte por SSH.
SERVIDOR WINDOWS (PowerShell)	Dentro del servidor Windows (o tu PC si instalas ahí mismo).
NAVEGADOR	Chrome / Edge / Firefox.

Regla de oro: un comando de Linux (bash) pegado en PowerShell da error, y al revés. **Mira siempre la etiqueta antes de pegar.**

0.4 Decisiones que tomarás durante la instalación

Ten claro esto desde ya (te lo preguntará el instalador):

1. **¿Cómo entrarán los usuarios?** Por **IP** (ej. `https://192.168.1.50`) — lo más simple, sin tocar DNS — o por **nombre** (ej. `https://watchlog.planta.interna`) si el cliente ya tiene un DNS interno.
2. **¿Modo de borde?**
 - **b = borde propio** (lo normal para una prueba/piloto): el instalador crea el certificado y la seguridad **solo**. Tú no tocas nada de certificados.
 - **a = detrás del proxy del cliente:** solo si el cliente YA tiene un F5 / NetScaler / NGINX / IIS que termina el HTTPS. Si dudas, usa **b**.

0.5 Anota estos datos (los usarás varias veces)

Dato	Ejemplo	El tuyo
IP del servidor	192.168.1.50	_____
Usuario del servidor	ubuntu / Administrador	_____
Contraseña / llave	(te la da el cliente)	_____
Versión del paquete	v0.1.21	_____

En esta guía se usa **v0.1.21** como ejemplo. **Reemplázala** por la versión que te entregaron (mira el nombre del archivo `.tar.gz`).

PARTE A — Instalar en LINUX

Sigue estos pasos **en orden**. Al final tendrás Lyra WatchLog funcionando en el navegador.

A1. Verifica que el servidor cumple los requisitos

- **Sistema:** Linux de servidor de 64 bits (**Ubuntu 22.04/24.04** o **Debian 12** recomendados; también RHEL/Rocky/Alma 8–9).
- **Arquitectura:** **x86-64 / amd64** (un PC/servidor Intel o AMD normal). **NO** sirve ARM (Raspberry Pi, servidores ARM).
- **Memoria:** **4 GB** mínimo (8 GB mejor). **Disco:** **10 GB** libres.
- **Reloj en hora** (importante para la seguridad y la auditoría).

✓ **Sabrás que puedes seguir** si tienes acceso al servidor y cumple lo de arriba.

A2. Conéctate al servidor e instala Docker

Docker es el **motor** que hace correr la aplicación.

TU PC (PowerShell) — conéctate por SSH (reemplaza usuario e IP):

```
ssh USUARIO@IP_DEL_SERVIDOR
```

- La **primera vez** te preguntará algo como `Are you sure...?` → escribe **yes** y Enter.
- Te pedirá la **contraseña** (al escribirla **no se ve nada**, es normal) → Enter.
- ✓ **Entraste** cuando el inicio de la línea cambia a algo como `usuario@servidor:~$`. **A partir de aquí estás DENTRO del servidor** (comandos bash).

SERVIDOR LINUX (bash) — instala Docker:

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER
```

Sal y vuelve a entrar para que el permiso tome efecto:

```
exit
```

TU PC (PowerShell) — reconéctate:

```
ssh USUARIO@IP_DEL_SERVIDOR
```

SERVIDOR LINUX (bash) — comprueba que quedó bien:

```
docker version
docker compose version
```

- ✓ **Bien:** cada uno muestra un número de versión.
- ✗ **Mal:** dice `command not found` → repite la instalación de Docker.

¿El servidor **NO** tiene internet? Pídele al cliente que instale Docker desde su repositorio interno. Es lo único que la app necesita de base.

A3. Copia el paquete al servidor

El paquete es **un solo archivo** de ~1 GB llamado `lyra-watchlog-v0.1.21.tar.gz`.

1. **NAVEGADOR (en tu PC):** descárgalo del sitio/USB que te indicó el proveedor (normalmente el **GitHub Release** del tag `v0.1.21`, sección **Assets**). Queda en tu carpeta **Descargas**.
2. **TU PC (PowerShell):** cópialo al servidor:

```
cd ~\Downloads
scp lyra-watchlog-v0.1.21.tar.gz USUARIO@IP_DEL_SERVIDOR:/home/USUARIO/
```

- La copia de 1 GB **tarda varios minutos** (verás una barra de progreso).
- **✓ Bien:** llega al 100 %.

¿Sin internet en el servidor? Descarga el `.tar.gz` en tu PC (con internet), pásalo a un **USB** y cópialo al servidor por ahí. El resultado es el mismo.

A4. Desempaqueta el paquete

TU PC (PowerShell): conéctate:

```
ssh USUARIO@IP_DEL_SERVIDOR
```

SERVIDOR LINUX (bash):

```
tar -xzf lyra-watchlog-v0.1.21.tar.gz
cd lyra-watchlog-v0.1.21
ls
```

- **✓ Bien:** ves archivos como `install.sh`, `doctor.sh`, `images`, `compose`, `SHA256SUMS`.

A5. Primera pasada del instalador

Esta pasada **carga la aplicación** y prepara la configuración. **Se detiene a propósito** para que completes dos datos.

SERVIDOR LINUX (bash):

```
./install.sh
```

Qué verás (todo **normal**): - **!** Un **aviso amarillo sobre cosign** ("firma NO verificada aquí"): **normal**, esa herramienta opcional no está en el servidor. La verificación importante (SHA256) **sí** corre. - El paso "**Verificación de integridad (SHA256SUMS)**" puede tardar **1–2 minutos** (revisa ~1 GB). El cursor quieto ahí **no** está colgado: espera. - **✓ Termina con un aviso** tipo "**COMPLETA lo específico del sitio en .env y vuelve a ejecutar**". **Eso NO es un error**: significa "ahora ve al paso A6".

A6. Dile a la app su dirección (edita SOLO .env)

Fija **dos** valores: el modo de borde y la dirección pública. Usa la **IP del servidor** (lo más simple). Estos dos comandos dejan las líneas **limpias, sin comentarios** (un comentario en la misma línea rompe el instalador):

SERVIDOR LINUX (bash) — reemplaza IP_DEL_SERVIDOR en el segundo comando:

```
sed -i 's/^EDGE_MODE=.* /EDGE_MODE=b/' .env
sed -i 's#^APP_PUBLIC_URL=.*#APP_PUBLIC_URL=https://IP_DEL_SERVIDOR#' .env
grep -E '^EDGE_MODE=|^APP_PUBLIC_URL=' .env
```

- **✓ Bien**: el grep muestra **exactamente** estas dos líneas, sin ningún # extra:

```
EDGE_MODE=b
APP_PUBLIC_URL=https://IP_DEL_SERVIDOR
```

- **✗ Si ves un # al final** de alguna línea: vuelve a correr los dos sed.

¿Tienes un certificado corporativo? (opcional, avanzado) Colócalo en certs/cert.pem (cadena completa) y certs/key.pem **antes** del paso A7; el instalador lo usará en vez del de prueba. Si no, no hagas nada: el instalador crea uno de prueba solo.

A7. Segunda pasada del instalador (enciende todo)

SERVIDOR LINUX (bash):

```
./install.sh
```

- **Crea el certificado de prueba** (si no pusiste uno), **genera** la config del borde desde tu dirección, **enciende** la app y **espera** a que esté "sana".
- **!** Verás "**Certificado SELF-SIGNED generado**": **normal** (por eso el navegador avisará luego que "la conexión no es privada").
- **✓ Bien**: al final dice **API saludable** y una lista de "próximos pasos".
- **✗ Si dice que la API NO quedó sana**: corre el **diagnóstico** (ver A11), que te dice exactamente qué falló.

En este punto la app **ya está corriendo**, pero en modo **solo lectura** hasta que le pongas la licencia (A8). Es **a propósito**: la licencia nunca borra ni secuestra datos.

A8. Ceremonia de licencia

Quién: el paso “generar” lo hace **el proveedor (ITESICWS)** en su PC con la llave de emisión. Tú solo mueves dos archivos. El servidor **no** necesita internet.

Al encender, el servidor dejó una **solicitud** con la “huella” de esta instalación. Flujo: **traer solicitud** → **generar licencia** → **devolver licencia**.

A8.1 — Trae la solicitud a tu PC TU PC (PowerShell):

```
scp USUARIO@IP_DEL_SERVIDOR:/home/USUARIO/lyra-watchlog-v0.1.21/license/solicitud.
↪ lreq .
```

- ✓ Ahora tienes `solicitud.lreq` en tu carpeta actual.

A8.2 — El proveedor genera la licencia (con su llave; ejemplo, todo en **una** línea):

```
pnpm license issue --request solicitud.lreq --customer "Cliente Prueba"
↪ --channel-partner PILOTO --edition enterprise --modules
↪ core,structure,templates,logbook,schedules,incidents,exceptions,work-orders,
↪ shift-handover,notifications,themes,ai,dashboards --max-nodes 50
↪ --max-named-users 25 --expires 2026-10-08T00:00:00Z --grace-days 14 --out
↪ license.lic
```

- ✓ Resultado: un archivo `license.lic`.

A8.3 — Devuelve la licencia al servidor TU PC (PowerShell):

```
scp license.lic USUARIO@IP_DEL_SERVIDOR:/home/USUARIO/lyra-watchlog-v0.1.21/license/
```

- ✓ La app toma la licencia **automáticamente** en unos segundos.

A9. Crea tu usuario administrador

La primera vez, la app te deja crear al administrador con un **código de un solo uso**.

A9.1 — Consigue el código TU PC (PowerShell):

```
ssh USUARIO@IP_DEL_SERVIDOR
```

SERVIDOR LINUX (bash):

```
cat lyra-watchlog-v0.1.21/license/setup-token
```

- ✓ Copia el texto largo que aparece (ese es tu código).

A9.2 — Abre el asistente NAVEGADOR: entra a (reemplaza la IP):

```
https://IP_DEL_SERVIDOR/setup
```


- El navegador mostrará **“la conexión no es privada”**: es **normal** (certificado de prueba). Haz clic en **“Avanzado”** → **“Continuar de todos modos”**.
- Pega el **código**, crea tu **correo y contraseña** de administrador, elige identidad/tema y confirma. Al terminar, el código se **desactiva solo**.

A10. Entra y prueba

NAVEGADOR: `https://IP_DEL_SERVIDOR` → inicia sesión con tu correo y contraseña. Recorrido de comprobación: 1. **Estructura** → crea un área/planta. 2. **Plantillas** → crea una plantilla con un par de campos. 3. **Bitácoras** → registra una entrada de turno. 4. **Incidencias** → abre una y muévela por su flujo. 5. **Configuración** → **Licencia** → debe decir **VÁLIDA**.

Si puedes hacer todo eso, **la instalación en Linux fue exitosa**.

A11. Si algo sale mal (diagnóstico)

Primer reflejo: corre el diagnóstico. Te da un reporte **PASA/FALLA** y convierte errores crípticos en una acción concreta.

SERVIDOR LINUX (bash): dentro de la carpeta del paquete:

```
./install.sh --check # (o ./doctor.sh)
```

Problemas comunes: - **“La API no quedó sana”**: casi siempre **poca RAM** (necesita ≥ 4 GB libres) o un puerto ocupado. El diagnóstico lo dirá. Revisa registros: `docker compose --project-directory . --env-file .env -f compose/docker-compose.yml -f compose/mode-b.own-edge.yml logs api` - **“La app queda en solo lectura”**: falta la licencia (A8) o no calza con este servidor (genera la licencia contra la solicitud.lreq de **este** servidor). - **La API no puede guardar la licencia (permisos)**: `sudo chown -R 1000:1000 license certs` y repite A8.3. - **El navegador no abre**: confirma `https://` (no `http://`) y que el cliente **abrió el puerto 443** en su firewall.

A12. Operación diaria (Linux)

SERVIDOR LINUX (bash), dentro de `lyra-watchlog-v0.1.21/`:

```
# Apagar (conserva Los datos):
docker compose --project-directory . --env-file .env -f compose/docker-compose.yml
↳ -f compose/mode-b.own-edge.yml down
# Encender de nuevo:
./install.sh
# Diagnóstico:
./install.sh --check
```

RespalDOS y actualización: ver docs/INSTALL_OFFLINE.md 9 y 10.

PARTE B — Instalar en WINDOWS

Windows corre la app dentro de **contenedores Linux** vía **Docker Desktop + WSL2**. El instalador `install.ps1` es el equivalente del `install.sh` de Linux. Sigue los pasos **en orden**.

Nota honesta: Windows es ideal para **pilotos/estaciones**. Para producción real, el destino recomendado es Linux (una capa menos).

B1. Verifica que el equipo cumple los requisitos

- **Sistema: Windows 10/11 Pro** de 64 bits (o Windows Server con Docker Desktop).
- **Arquitectura: x86-64 / amd64** (Intel/AMD). **NO** sirve Windows ARM.
- **Virtualización ACTIVADA** en la BIOS/UEFI (Intel VT-x / AMD-V) — Docker Desktop la necesita. Suele venir activada; si no, se activa en la BIOS.
- **Memoria: 8 GB** recomendado. **Disco: 10 GB** libres.

B2. Instala WSL2

WSL2 es el que aporta el **kernel de Linux** donde corren los contenedores.

SERVIDOR WINDOWS (PowerShell como Administrador) — clic derecho en PowerShell → “Ejecutar como administrador”:

```
ws1 --install
```

- Instala WSL2. **Reinicia el equipo** cuando lo pida.
- **✓ Comprueba** tras reiniciar:

```
ws1 --status
```

- **✗** Si dice que la virtualización no está activada → actívala en la BIOS y repite.

B3. Instala Docker Desktop (en modo contenedores Linux)

1. **NAVEGADOR:** descarga **Docker Desktop** desde <https://www.docker.com/products/docker-desktop/> e instálalo (deja marcada la opción “**Use WSL 2 based engine**”).
2. Abre Docker Desktop y espera a que el ícono de la **ballena** (abajo a la derecha) quede **estable** (Docker running).
3. **Asegura el modo LINUX:** clic derecho en el ícono de la ballena → si aparece “**Switch to Linux containers...**”, ese texto significa que **estás en modo Windows** → **haz clic para cambiar a Linux**. Si aparece “Switch to **Windows** containers”, ya estás en **Linux** (correcto, no toques nada).
4. **✓ Comprueba: SERVIDOR WINDOWS (PowerShell):**

```
docker version -f '{{.Server.Os}}/{{.Server.Arch}}'
```

- **✓ Bien:** imprime `linux/amd64`.
- **✗ windows/...** → vuelve al punto 3 y cambia a **Linux containers**.

B4. Instala OpenSSL

El instalador usa **openssl** para crear los secretos y el certificado (igual que en Linux).

1. **NAVEGADOR:** descarga **“Win64 OpenSSL Light”** desde <https://slproweb.com/products/Win32OpenSSL.html> e instálalo. En el instalador, cuando pregunte dónde copiar las DLLs, elige **“The Windows system directory”** (así queda en el PATH).
2. **Cierra y abre PowerShell** (para que tome el PATH nuevo) y comprueba: **SERVIDOR WINDOWS (PowerShell):**

```
openssl version
```

- **✓ Bien:** imprime algo como `OpenSSL 3.x . . .`.
- **✗ “no se reconoce openssl”** → reinstala marcando la copia al directorio del sistema, o agrega manualmente `C:\Program Files\OpenSSL-Win64\bin` al **PATH** y reabre PowerShell.

B5. Descarga y desempaqueta el paquete

1. **NAVEGADOR:** descarga `lyra-watchlog-v0.1.21.tar.gz` (GitHub Release / USB del proveedor).
2. **SERVIDOR WINDOWS (PowerShell):** crea una carpeta **de ruta corta** y desempaca ahí (Windows 10/11 traen tar incluido):

```
cd C:\
tar -xzf $HOME\Downloads\lyra-watchlog-v0.1.21.tar.gz -C C:\
cd C:\lyra-watchlog-v0.1.21
dir
```

- **✓ Bien:** ves `install.ps1`, `doctor.ps1`, `images`, `compose`, `SHA256SUMS`.

Recomendado (rendimiento y permisos): si puedes, trabaja dentro del disco de **WSL2** (ext4) en vez de `C:\`. Para una prueba, `C:\` funciona; si el paso B7 avisa de permisos, mira B10.

B6. Primera pasada del instalador

SERVIDOR WINDOWS (PowerShell):

```
.\install.ps1
```

- **✗ Si dice que la ejecución de scripts está deshabilitada** (“no se puede cargar el archivo ... `install.ps1`”), corre así (permite el script **solo en esta ventana**):

```
powershell -ExecutionPolicy Bypass -File .\install.ps1
```

Qué verás (**normal**): - Preflight en **linux/amd64**, verificación de integridad (**1–2 min**, no está colgando), carga de imágenes, creación del `.env` con secretos, y **la huella anclada a tu PC** (MachineGuid).
 - **!** Aviso de cosign “no verificado aquí”: **normal** (opcional). - **✓ Termina** pidiendo completar `EDGE_MODE + APP_PUBLIC_URL`. **No es error**: ve a B7.

B7. Dile a la app su dirección (edita SOLO .env)

Usaremos la **IP del equipo** (mírala con `ipconfig` si no la sabes). Estos comandos dejan las líneas limpias:

SERVIDOR WINDOWS (PowerShell) — reemplaza `IP_DEL_SERVIDOR`:

```
(Get-Content .env) -replace '^EDGE_MODE=.*', 'EDGE_MODE=b' -replace
↪ '^APP_PUBLIC_URL=.*', 'APP_PUBLIC_URL=https://IP_DEL_SERVIDOR' | Set-Content .env
Select-String -Path .env -Pattern '^EDGE_MODE=|^APP_PUBLIC_URL='
```

- **✓ Bien**: muestra exactamente:

```
EDGE_MODE=b
APP_PUBLIC_URL=https://IP_DEL_SERVIDOR
```

- **! NO agregues comentarios (#)** en esas dos líneas: rompen el instalador.

B8. Segunda pasada del instalador (enciende todo)

SERVIDOR WINDOWS (PowerShell):

```
.\install.ps1
```

- **Genera** el certificado de prueba + la config del borde, hace un **probe de permisos** (que la app pueda escribir su licencia), **enciende** y **espera** a que esté “sana”.
- **!** “Certificado SELF-SIGNED generado”: **normal**.
- **✓ Bien**: al final dice **API saludable** y los “próximos pasos”.
- **✗ Si el probe de permisos falla**: ve a **B10** (File Sharing / mover a WSL2).
- **✗ Si la API no quedó sana**: corre el diagnóstico (B11).

B9. Licencia + administrador + prueba

Esto es **igual** que en Linux; solo cambian los comandos por su versión PowerShell.

B9.1 — Licencia (el proveedor la genera con su llave; el servidor no necesita internet): **SERVIDOR WINDOWS (PowerShell)**: la solicitud quedó en `license\solicitud.lreq`. Envíala al proveedor (por correo/USB); te devuelven **license.lic**. Cópialo a la carpeta `license\`:

```
# (si lo recibes en Descargas, por ejemplo)
Copy-Item $HOME\Downloads\license.lic .\license\
```

- ✓ La app toma la licencia en unos segundos.

B9.2 — Administrador (código de un solo uso): SERVIDOR WINDOWS (PowerShell):

`Get-Content .\license\setup-token`

NAVEGADOR: entra a `https://IP_DEL_SERVIDOR/setup` → “**Avanzado**” → “**Continuar de todos modos**” (certificado de prueba) → pega el **código** → crea **correo y contraseña** de administrador → confirma.

B9.3 — Entra y prueba: NAVEGADOR: `https://IP_DEL_SERVIDOR` → inicia sesión y recorre **Estructura** → **Plantillas** → **Bitácoras** → **Incidencias** → **Configuración/Licencia (VÁLIDA)**.

Si puedes hacer todo eso, **la instalación en Windows fue exitosa**.

B10. Si el probe de permisos falla (Windows)

El instalador avisa si la app (que corre **sin privilegios**) **no puede escribir** su carpeta de licencia a través del puente de Docker. Soluciones (en orden): 1. **Habilita la unidad en Docker Desktop:** Settings → **Resources** → **File Sharing** → agrega la unidad/carpeta del paquete (p. ej. C:\) → **Apply & Restart**. Reejecuta `.\install.ps1`. 2. **Mueve el paquete al disco de WSL2** (más rápido y sin este problema): abre tu distro de WSL (`wsl`) y desempaca/instala el paquete dentro de tu carpeta de Linux (`~/`). Ahí los permisos son nativos.

B11. Si algo sale mal (diagnóstico Windows)

SERVIDOR WINDOWS (PowerShell): dentro de la carpeta del paquete:

```
.\install.ps1 -Check      # (o .\doctor.ps1)
```

Da un reporte **PASA/FALLA** (arquitectura, Docker, openssl, puertos, certificado, salud de cada contenedor). Problemas comunes: - **docker version dice windows/...** → cambia a **Linux containers** (B3). - “**no se reconoce openssl**” → instala OpenSSL y reabre PowerShell (B4). - **Puerto 443/80 ocupado** → hay otro servidor web (IIS, otro Docker). Libéralo o cámbialo. - **La app queda en solo lectura** → falta la licencia (B9.1) o no calza con este equipo.

B12. Operación diaria (Windows)

SERVIDOR WINDOWS (PowerShell), dentro de la carpeta del paquete:

Apagar (conserva datos):

```
docker compose --project-directory . --env-file .env -f compose\docker-compose.yml
↪ -f compose\mode-b.own-edge.yml down
```

Encender de nuevo:

```
.\install.ps1
```

Diagnóstico:

```
.\install.ps1 -Check
```

Apéndice — Lista de verificación final (ambos sistemas)

Marca cada punto; si todos están , **terminaste**:

- ☐ Docker responde (docker version OK). En Windows: docker version -f '{{.Server.Os}}' = **linux**.
- ☐ El paquete se desempacó y ves install.sh / install.ps1.
- ☐ 1ª pasada del instalador OK (creó .env, se detuvo pidiendo datos).
- ☐ .env con EDGE_MODE y APP_PUBLIC_URL **sin comentarios** al final.
- ☐ 2ª pasada del instalador → **API saludable**.
- ☐ Diagnóstico (--check / -Check) → **TODO EN VERDE**.
- ☐ Licencia aplicada (license.lic copiado) → la app **no** está en solo lectura.
- ☐ Administrador creado en /setup.
- ☐ Inicias sesión en https://<IP> y recorres los módulos.
- ☐ **Configuración** → **Licencia** dice **VÁLIDA**.

Glosario mínimo

- **Docker / contenedor**: el "motor" que corre la app aislada, sin instalar nada suelto.
- **.env**: el archivo de configuración del sitio (dirección, secretos). El instalador lo crea.
- **Modo de borde (a/b)**: quién pone el HTTPS. b = lo pone Lyra (lo normal); a = el proxy del cliente.
- **Certificado self-signed**: certificado de prueba; el navegador avisa "no es privada" — es esperado en pilotos.
- **solicitud.1req / license.lic**: la "huella" del servidor y la licencia firmada que el proveedor genera a partir de ella.
- **/setup + setup-token**: el asistente de primer arranque para crear al administrador.
- **doctor (--check / -Check)**: diagnóstico PASA/FALLA; tu primer reflejo ante cualquier problema.

Para profundizar

- docs/INSTALL_OFFLINE.md — guía técnica (respaldos cifrados, actualización offline, modos de borde avanzados, verificación de firma). Windows en su **12**.
- docs/PRUEBA_PILOTO.md — guía de piloto para principiantes (Linux, con variante Windows).
- docs/SUPPORTED_PLATFORMS.md — qué sistemas operativos y arquitecturas están soportados.
- docs/LICENSING_PROCEDURE.md — la ceremonia de licencia en detalle.